
Kandidatarbete i medieteknik, Institutionen för teknik och estetik, vårtermin 2025

Satellite-driven environment reconstruction in Unreal Engine

A simplified process for widespread adoption



Authors:

Jimmy Johansson

André Riis

Supervisors: Markus Fiedler | Sejke Nord | Andrea Nordwall | Linus de Petris
Examiner: Markus Fiedler

Abstract

Photorealistic 3D environments are widely used in film, architecture, and AI training, yet recreating real-world locations using satellite data often requires expert knowledge and extensive timeconsuming manual effort. This limits the accessibility and range of applications for the technology. This bachelor's project aims to simplify the process by developing a user-friendly tool in Unreal Engine that automates environment generation using satellite imagery and elevation data. The tool is designed for users with only basic experience, while still offering advanced configuration options for more experienced users.

The study follows an iterative design method, testing and refining technical solutions. The outcome is a functional prototype that generates landscapes, environments and vegetation with minimal manual input. The project demonstrates time savings and outlines the potential for further development of automated pipelines for reality-based 3D environments.

Keywords

Environment generation, Satellite imagery, Unreal Engine, Blueprint, Procedural content generation, PCG Biomes, AI Segmentation

Abstrakt

Fotorealistiska 3D-miljöer används inom film, arkitektur och AI-träning, men processen för att återskapa verkliga platser baserat på satellitdata kräver ofta expertkunskap och omfattande tidskrävande manuellt arbete. Detta begränsar teknologins tillgänglighet och användningsområden. Detta kandidatarbete syftar till att förenkla denna process genom att utveckla ett användarvänligt verktyg i Unreal Engine som automatiserar miljögenerering med hjälp av satellitbilder och höjddata. Verktuget kräver endast grundläggande användarkunskaper, men erbjuder även avancerade inställningar för erfarna användare.

Studien bygger på en iterativ designmetod där tekniska lösningar testas och förfinas. Resultatet är ett fungerande prototypverktyg som genererar landskap, miljöer och vegetation med minimal manuell insats. Projektet visar på tidsbesparingar och möjligheter till vidareutveckling av automatiserade pipelines för verklighetsbaserade 3D-miljöer.

Nyckelord

Miljö generering, Satellitbilder, Unreal Engine, Blueprint, Proceduellt genererat innehåll, PCG Biomes, AI Segmentering

Contents

Abstract.....	1
Keywords	1
Abstrakt.....	1
Nyckelord.....	1
Terminology.....	4
1. Introduction.....	5
2. Related Work	6
2.1 Academic studies	6
2.2 Hobby and Community Projects.....	6
2.2.1 Unreal Engine Forum: DEM Data and Splatmap.....	6
2.2.2 Project Atlante: Earth-scale worlds	6
2.2.3 Photoreal Environments from Scratch: Transmedia & Film/TV.....	6
2.3 Summary	6
3. Methods	7
3.1 Research Methods	7
3.2 Design Methods	7
3.2.1 Choice of Platform: Unreal Engine 5	7
3.2.2 Prototyping and Iteration Process	7
3.2.3 Evaluation Criteria	8
3.2.4 Iterative Refinement.....	8
3.3 Project Methods	8
3.4 Connection to the Research Question	8
4. Results	9
4.1 Use case	9
4.2 Tool functionality and usage	9
4.3 Pilot study	9
4.4 Technical limitations	10
4.5 Heightmap conversion	10
4.5.1 Limitations of Blueprint for heightmap implementation	10
4.5.2 Python-based solution for 16 bit-heightmap conversion.....	10

4.5.3 Challenges with landscape size matching	10
4.6 Weather and material effects.	10
4.7 From Satellite Data to Usable Textures in Unreal Engine	11
4.7.1 Tool Overview	11
4.7.2 Dataset and Training Procedure	11
4.7.3 Inference and Texture Generation	11
4.7.4 Interactive Developer Mode.....	12
4.7.5 Technical Implementation and Dependencies	12
4.7.6 Post Processing Pipeline.....	12
4.8 Landscape AutoMaterial with Satellite Image Overlay	13
5. Discussion.....	13
5.1 Automation and Future Improvements	13
5.2 Technical Limitations: Heightmap Handling	14
5.3 User Interface and Simulation Realism	14
5.4 PCG Biomes and Landscape Coverage	14
5.5 PCG Generation Graphs	15
5.6 Development Constraints and Editor Limitations.....	15
5.7 AI-Based Segmentation of Satellite Imagery	16
5.7.1 Model Design and Training.....	16
5.7.2 Effectiveness of the Post Processing Pipeline.....	16
5.7.3 Opportunities for Refinement.....	16
5.8 Data Source Evaluation and Satellite Imagery Quality	16
6. Conclusion	17
7. Future Work	17
8. References.....	19
9. Appendix.....	21

Terminology

Biomemap – Also called splatmap or biometexture is a texture where each color defines a different environment.

Blueprint – Unreal Engines own visual programming language

Heightmap – A grey scale texture which defines the terrains elevation, where brighter colors mean that it will be raised more

PCG – Procedural content generation, a system in unreal that allow a user to define logic for large tasks, for example placing trees on a large landscape.

PCG Biomes – A complex PCG subsystem that uses Unreal engine to place assets based on a biomemap.

Unreal Engine – Sometimes shortened to UE is a graphics engine that focuses on high fidelity. Version 5.0 or Unreal Engine 5 was released in 2020.

Digital Elevation Model (DEM) – A raster map where each pixel represents elevation, used to create topographic terrain in GIS and game engines.

1. Introduction

Photorealistic, georeferenced 3D environments are used today in everything from film production and architecture to training autonomous systems. Despite their potential benefits, recreating a real-world site in a game or visualization engine still demands significant expertise and manual effort-especially when working with satellite data and other geospatial information. The task is therefore both time- and cost-intensive, limiting the technology's availability and the range of possible applications. Several studies also highlight an ongoing gap in methods and tools for automatically converting geospatial datasets into high-quality 3D scenes.

This limitation has practical consequences for industries that could benefit from the technology but lack budget or specialist skills. For example, resource-constrained film studios, architects who need to rapidly evaluate design options in their real context, and navigation-AI developers are prevented from integrating realistic environments into their workflows. Unlocking the technology's full potential thus requires tools that simplify and automate the workflow while preserving the level of detail and quality demanded by professional users.

The purpose of this bachelor's project is to develop and evaluate a tool that automates as many steps as possible in creating a photorealistic 3D world based on satellite data in Unreal Engine, making the technology accessible even to users with limited 3D-production experience.

To meet this goal, the study poses the following research question:

How can the process of creating a reality-based 3D environment be simplified by using satellite data?

To keep the scope manageable, the work is limited to Unreal Engine as the graphics engine-because its built-in support for large-scale landscapes and photorealistic rendering is more mature than comparable tools-satellite and elevation data covering a restricted geographic area, and a target audience of users with a basic understanding of 3D environments but no requirement for expert knowledge of GIS or rendering pipelines.

The expected contribution is a functioning tool that lets inexperienced users generate a base environment with just a few clicks, while also giving advanced users options to adjust parameters to meet specific needs. Beyond the tool itself, the project delivers an empirical evaluation of time savings and usability, along with design principles that can guide future development of similar systems.

The article is structured as follows: After this introduction comes the Related Work section, which analyzes prior research and existing tools. The Method section then describes the research, design, and project methods applied. Results present the tool's functionality and empirical data, which are critically discussed in the Discussion section. Finally, the Conclusion summarizes the main findings and identifies possible future work, followed by References and any Appendices.

2. Related Work

2.1 Academic studies

Gross et al. (2022) demonstrate how elevation data from satellite/remote sensing can be utilized to create a coastal environment in Unreal Engine 5. They describe methods to avoid problematic geometry when importing mesh files directly, as well as how to add trees and bushes that move realistically. They provide a ready-to-use blueprint, although several tasks-such as choosing appropriate plant species and fine-tuning textures-still require manual input.

There are currently very few peer-reviewed studies that integrate satellite data and game engines into a fully automated pipeline. Most research is focused either on GIS visualization or offline rendering. This highlights the need for research that simplifies the process for non-experts aiming to run realtime environments, which is precisely the gap our research aims to address.

2.2 Hobby and Community Projects

2.2.1 Unreal Engine Forum: DEM Data and Splatmap

In a post on the Unreal Engine forum, user sanford87 (2020-10-10) describes how Digital Elevation Model (DEM) data and satellite images are combined in ArcGIS Pro to generate a 50 km² landscape. A support vector machine is then used to generate a splat map from satellite imagery. After generating the landscape, performance is estimated at around 120 fps. However, this process still requires substantial manual input and adjustment (sanford87, 2020).

2.2.2 Project Atlante: Earth-scale worlds

Recently uploaded lectures and tutorials from Unreal Fest 2024 demonstrate the use of map data to create Earth-scale worlds with placed environments roughly matching real-world settings. The presentation "Project Atlante: An Experiment in Building Earth-Scale Worlds with Unreal Engine" details various techniques. Compared to our work, Project Atlante emphasizes larger scale but lower quality and precision (Epic Education et al., 2024a)

2.2.3 Photoreal Environments from Scratch: Transmedia & Film/TV

Additionally, the session "Creating a Photoreal Environment from Scratch for Transmedia & Film/TV" illustrates the creation of photorealistic worlds using satellite data extracted via the Unreal Engine 5 plugin named Cesium. The presenters apply materials that project geometry heights using a grayscale scale. They then take satellite imagery to generate a heightmap, subsequently recreating the environment in greater detail (Epic Education et al., 2024b)

2.3 Summary

The scientific article we reviewed is thorough, yet their workflows still require numerous manual adjustments. The hobby projects present clever shortcuts that function effectively but lack scientific validation. None of these methods deliver a complete, user-friendly process capable of transforming raw data into a fully functional 3D environment without manual intervention.

Our research aims to further bridge this gap by significantly reducing manual efforts. We develop and test a method allowing terrain, textures, and vegetation to be fetched and applied directly with a single

click. We measure both time savings and the outcome to demonstrate the practical advantages of our approach.

3. Methods

This section describes the methods used to evaluate and iteratively improve our pipeline for satellite data integration in a 3D environment. The section is divided into three main parts - research methods, design methods, and project methods - and concludes with a discussion on how the chosen methods relate to our research question.

3.1 Research Methods

Our overall approach is a mixed-methods design within an iterative Design-Based Research model (Easterday, Lewis, & Gerber, 2014). Technical performance - computation time, frame rate, and AI segmentation accuracy - was measured quantitatively through controlled experiments in Unreal Engine and Python scripts, while the tool's usability was evaluated qualitatively through informal observations and self-assessments during family testing. All data collection was conducted internally, without external participants or formal recruitment, which eliminated the need for ethical approvals or anonymization.

The quantitative metrics were taken directly from the tool's profiling and logs, without subsequent statistical testing, and the qualitative observations were documented anecdotally without thematic coding or formal content analysis. This choice enabled fast data collection in each iteration of the build-measure-learn cycle.

3.2 Design Methods

3.2.1 Choice of Platform: Unreal Engine 5

Unreal Engine 5 was selected as the primary development environment due to its robust support for open-world workflows and high-fidelity rendering features. Key technologies such as Nanite (Epic Games, n.d.) and Lumen (Epic Games, n.d.) enable highly detailed geometry and dynamic global illumination without significant performance trade-offs. These features are particularly suited for projects involving procedural generation and satellite data integration, where visual realism and scalability are critical factors. The platform's ecosystem, including landscape tools, material systems, and external plugin support, made it a natural fit for our iterative prototyping process (Misztal, 2024).

3.2.2 Prototyping and Iteration Process

We applied the Lean Startup methodology (Build-Measure-Learn) as the framework for our prototyping and iterative design process. In each cycle, a minimal version of the *Procedural Content Generation-Biomes* (PCG-Biomes) tool was built in Unreal Engine 5. In parallel, an external AI-based Python script was executed for satellite image segmentation. Each iteration focused on a tightly scoped feature set, enabling fast feedback and adaptation.

3.2.3 Evaluation Criteria

After each build, the system's performance and internal usability were evaluated against predefined criteria:

- **Performance:** Time to generate each biome and other system metrics.
- **Accuracy:** Proportion of correctly classified pixels compared to a "true" segmentation mask.
- **Internal Usability:** Degree of simplicity and workflow alignment with project needs.

These metrics allowed for objective assessments of system capability, while also identifying areas for refinement.

3.2.4 Iterative Refinement

When the results of a cycle did not meet internal benchmarks, the shortcomings were analyzed. Algorithm settings, system parameters, and user flows were revised based on observed deficiencies. The next iteration would then incorporate these changes and re-enter the Build-Measure-Learn loop. This process was repeated until acceptable performance and usability thresholds were achieved.

It should be noted that these internal benchmarks were based solely on our subjective ratings and experiences rather than formal benchmarking. Certain evaluations, including tests of mapping software and AI models, are documented in Appendix 8.

3.3 Project Methods

The project followed an agile-inspired Kanban-like workflow without fixed sprints. Tasks were prioritized and continuously moved between "To do", "Doing", and "Done" in a shared workspace. Roles and responsibilities were clearly divided, with some overlap to support each other with difficult tasks:

- Jimmy (Environment Lead): Development of AI scripts, landscape materials, implementation of weather effects, and environment design.
- André (Tool Lead): Design and implementation of the Unreal tool, including user interface, workflow logic, landscape generation and programming.

Timelines and milestones were broadly defined for each iteration, with ongoing check-ins rather than fixed deadlines. Six main risks were identified (performance, optimization, realism, time consumption, data sources, complexity) and addressed through technical optimizations (Nanite, Virtual Textures), downscaling of detail levels, use of a web-based PNG heightmap tool complemented by a custom AI tool, and automation of critical steps. Motivation for some of these choices will be brought to light later in this paper.

3.4 Connection to the Research Question

Through the internal Build-Measure-Learn process, we gained insights into which parts of the pipeline flow could be automated or simplified, directly addressing our research question of how satellite data integration can streamline the creation of realistic 3D environments. Experiments with various data sources (Cesium Sky, ArcGIS Maps SDK, Map Bridge, web-based PNG heightmap) clearly showed how data quality and manual effort affect the workflow. Technical optimization using Nanite and Virtual Textures, as well as automation of manual steps, exemplifies how the method enables fast and

simple biome generation - in line with our aim to test and validate how satellite data can best simplify the process.

4. Results

4.1 Use case

There are currently very few tools available to recreate detailed environments from real world data. We want to get as close as possible to creating a tool that anyone could use to create an environment with a decent amount of detail no matter their experience. As an example, this could be used for planning a construction project.

4.2 Tool functionality and usage

We have achieved a working prototype that is able to generate a landscape with different environments based on a texture supplied by the user. The complexity of the environment along with other variables affect the time it takes to generate the environment, but it is usually done within a few minutes. In the simplest configuration of the tool there are only three inputs, a heightmap texture, a biome map texture and landscape scale, the landscape scale can even be skipped, but it will come at the cost of accuracy compared to the original heightmap. Currently the tool will prompt the user to input these into the minimalistic user interface and will show warnings if they are missing. Both the heightmap and landscape scale are obtained from the same website, which also supplies the satellite image for the selected area.

There is also an advanced part of the tool that is as standard hidden behind a button titled “advanced”, this part of the tool provides the user with the ability to choose specifically which biomes they want to be placed. Beyond that it also allows them to add their own assets to be placed by biomes and change the settings for that asset, like for example placement amount and how much space the asset will take up in the perspective of the PCG system, larger sizes will mean that the asset reserves more space around itself.

4.3 Pilot study

A small-scale pilot study was conducted to test how user-friendly our interface was. In this study the users are assumed to already have acquired and imported the biometexture, heightmap and landscape scale. They are given a note with the names of the pre-imported files and the landscape scale and are then left alone to try and use the tool to generate an environment using the defined items. None of the users in the study had any prior experience with 3D software or graphics engines.

The main issues noted by users was both a lack of direct instructions on where to click to input your items as well as navigating Unreal's own interface, for example when using the search bar to find your heightmap, Unreal does not sort the results based on likeness but rather displays all of the results unsorted, forcing the user to scroll through the list to try and find the correct file. Another issue was the additional buttons added by Unreal called “Use selected asset from content browser” and “Browse to asset in content browser”. These are useful for developers but cause confusion for regular users.

4.4 Technical limitations

Because of the workarounds we had to create there are some current technical limitations. One of the limitations is the size of the landscape, a larger landscape means a lot more assets must be placed, and some biomes require a lot more assets, for example the farmland biome requires a lot more points to be generated compared to forest. Going over a certain landscape size means that the engine can't quite handle the number of points and will freeze and eventually crash. Using lower complexity biomes means that you can create larger landscapes. We have currently tried using resolutions up to 4033x4033 which is equivalent to a 10.5x10.5km area, this takes a decent amount of time to generate so we assume that going a step higher, which would be 8129x8129 (136x137km) is going to just crash the engine without major optimizations.

4.5 Heightmap conversion

4.5.1 Limitations of Blueprint for heightmap implementation

There is currently no way to create a landscape directly from Blueprint, so we had to rely on a plugin called TAPython that has a node which allows creation of landscapes from the code but it also has a function that allows you to apply height data from a unsigned 16 bit-array to a landscape. We originally tried Unreal's own apply heightmap from render target node but after a lot of debugging we learned that unreal won't write 16 bit-textures to a render target, only 8 bit, so all the landscapes ended up looking blocky due to the low data precision.

4.5.2 Python-based solution for 16 bit-heightmap conversion

We solved this by creating a python script that can convert a Png heightmap into an unsigned 16 bit-integer array using the OpenCV python package. By utilizing string variables and unreal's support for python we are able to run the script directly from our Blueprint code. We grab the source file path from the imported texture from unreal since unreal converts imported textures to the .uasset format we must get the original file from the disk, after grabbing the source file path we feed it to our script. After the python script reads the file, it exports it as a csv file where each pixel is equal to an unsigned 16 bit-integer (Range: 0-65535) that can be imported to a data table in Unreal.

4.5.3 Challenges with landscape size matching

Another major issue is the landscape size, the problem being that the landscapes need to match the array size from the imported heightmap csv file exactly or it won't be able to apply the heightmap array to the landscape. Due to some current issues, we have it hardcoded to work with a couple of Unreal's recommended landscape sizes. The most notable supported sizes and what scale they represent in the real world are 4033x4033 (10.5x10.5km), 2017x2017 (5.5x5.5km) and 1009x1009 (2.7x2.7km).

4.6 Weather and material effects.

We employed the Ultra Dynamic Sky plugin to simulate weather, wind, and lighting. To streamline integration, we created simplified material functions that let us apply these effects to all scene assets quickly. As a result, every object responded coherently to the same weather scenario; snowfall covered surfaces with snow, rain made them wet, and wind caused them to move. The material functions included in the plugin worked but were time consuming to implement, whereas our

approach reduced implementation time substantially. Runtime virtual textures were also added to create a blending effect between the landscape and assets. The results can be seen in Figure 1.



Figure 1 Environment with snow effects.

4.7 From Satellite Data to Usable Textures in Unreal Engine

4.7.1 Tool Overview

Our independent Python tool, developed with early guidance from OpenAI o1, converts raw satellite imagery into the color and mask data required by the procedural generation workflow in Unreal Engine.

4.7.2 Dataset and Training Procedure

We train the model on a private image set that we assembled. For every land class such as forest, farmland, and scrub vegetation we manually labeled 10-20 representative satellite image snippets. During training each snippet is rescaled to 32×32 pixels and the images are normalized to RGB floats. When the program starts it computes an MD5 hash of the training folder. If the data have changed it trains a compact three-layer neural network with about 35 000 parameters in PyTorch on the GPU; if not, it loads the most recent validated network within seconds.

4.7.3 Inference and Texture Generation

During inference the satellite image has been verified at resolutions up to 8129×8129 pixels without memory issues. The image is converted to normalized RGB values, and every pixel is classified. The program then produces two full resolution outputs. The first output is a biome color map that can be used directly as the biome texture in Unreal Engine. The second output is an individual 8-bit grayscale mask for every class; these masks are intended to drive landscape material assignment, for example sending the forest mask to the forest ground material. An optional Gaussian blur can soften mask boundaries when desired. The different masks can be seen in Figure 2.



Figure 2 The original satellite image is shown alongside the texture layers generated by our AI tool.

4.7.4 Interactive Developer Mode

A developer mode lets users zoom and pan within the color map, mark misclassified regions, add new training snippets and trigger an immediate retrain. Because the network is small, the entire cycle completes in a few seconds, and the updated results appear instantly.

4.7.5 Technical Implementation and Dependencies

The code relies on PyTorch for neural network operations, Torchvision for data utilities, NumPy for array processing, Pillow for image input and output, OpenCV for fast image filtering, Matplotlib for interactive inspection, Scikit Learn for label encoding, and standard libraries such as os, hashlib, and pickle for file handling, hashing, and model caching. These components together provide a fast, scalable, data driven bridge between satellite imagery and semantically precise landscape data and form a central element in our automated real time creation of large virtual environments.

4.7.6 Post Processing Pipeline

To refine the raw segmentation output and prepare it for production use in Unreal Engine, we developed four additional scripts. These scripts are executed sequentially after the initial AI-based segmentation and form a complete post processing chain.

4.7.6.1 Morphological Filtering and Class Cleanup

The first step applies a series of GPU-accelerated morphological operations and statistical filters to improve the quality of the class masks. It removes small, isolated misclassifications and smooths boundary transitions between different biome types. This can be seen in Figure 3.

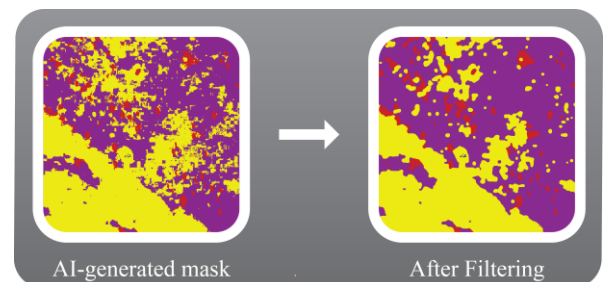


Figure 3 Mask before and after morphological filtering.

4.7.6.2 Road Network Extraction from Grayscale Input

The second step processes a separate grayscale road map in which roads are already visually highlighted. Rather than inferring road positions from the same satellite image used for segmentation, this step relies on contrast-enhanced input. It performs water removal, binarization, skeletonization, and geometric filtering to generate a refined road mask. This mask is later combined with the other biome layers.

4.7.6.3 Composite Texture Generation and File Structuring

In the third step, all cleaned class-specific masks are merged into a single RGB biome texture. Each biome class is represented by a distinct color, and the rendering order ensures that roads are displayed above other layers. This stage also creates updated binary masks based on the final composite and consolidates all outputs into a structured export folder.

4.7.6.4 Resolution Alignment for Terrain Compatibility

The final step resizes all exported class masks to a fixed target width while preserving their original aspect ratio. This ensures that all masks match the resolution of other landscape inputs, such as the heightmap. Misaligned resolutions can cause conflicts in Unreal Engine's landscape material system, particularly when combining grayscale masks with procedural features. To avoid these issues, the script resizes only the masks while preserving full resolution in the biome texture and satellite albedo.

4.8 Landscape AutoMaterial with Satellite Image Overlay

We purchased a *Landscape AutoMaterial* (Frenzel, 2025) to save time and avoid the complex work of creating satellite-based materials with multiple texture maps to cover the ground and add detail. Its key feature is the satellite image overlay, which adds color variation while maintaining detailed textures. This tool streamlined landscape creation with minimal manual effort.

5. Discussion

5.1 Automation and Future Improvements

There seems to be very little research done in this specific area which is recreation of real environments using satellite data. There is a lot that could be improved in the current prototype, and many parts still require testing and refinement. It is considered possible to automate even more features, such as the processing, importing, and application of the downloaded satellite data.

We limited our work to only include a few biomes, it's currently possible to generate forests, farmland and grass fields. These biomes are in line with what our AI can currently detect with good accuracy in images. Adding more of both biomes and data to the AI is time consuming and out of the scope for this project. Creating more biome presets is done by choosing what assets you want to be included and then setting the generator for them to use, a lot of the time generators can be reused, in cases like making different forest types but creating new generators means spending a lot of time programming in the PCG system.

The biome presets we are currently using have been of visually good quality but are a generic version of the type of area they represent. For example, we currently only define a forest as a forest, not a coniferous forest or broadleaf forest, the same goes for our farmland. This is because of the limitations in our AI training to detect the differences between these environments. With better data we can create vastly more varied and natural environments.

5.2 Technical Limitations: Heightmap Handling

The heightmap has been a cause for quite a lot of the technical limitations and workarounds since the goal was to create the landscape and apply the heightmap directly from the tool instead of doing it in Unreal's own hardcoded way. This approach works but has a few problems. Unreal does not support 16 bit-integers, which means that for every pixel in the file, it creates a 32 bit-integer. This takes up significantly more space and causes a 4096x4096 PNG file to become 1.6 GB in size when imported to the data table. Trying to fill an array or apply that array to the landscape causes the entire engine to freeze for a couple of minutes but it is possible to generate a landscape using it. A 10.5x10.5km landscape can be seen in Figure 4.

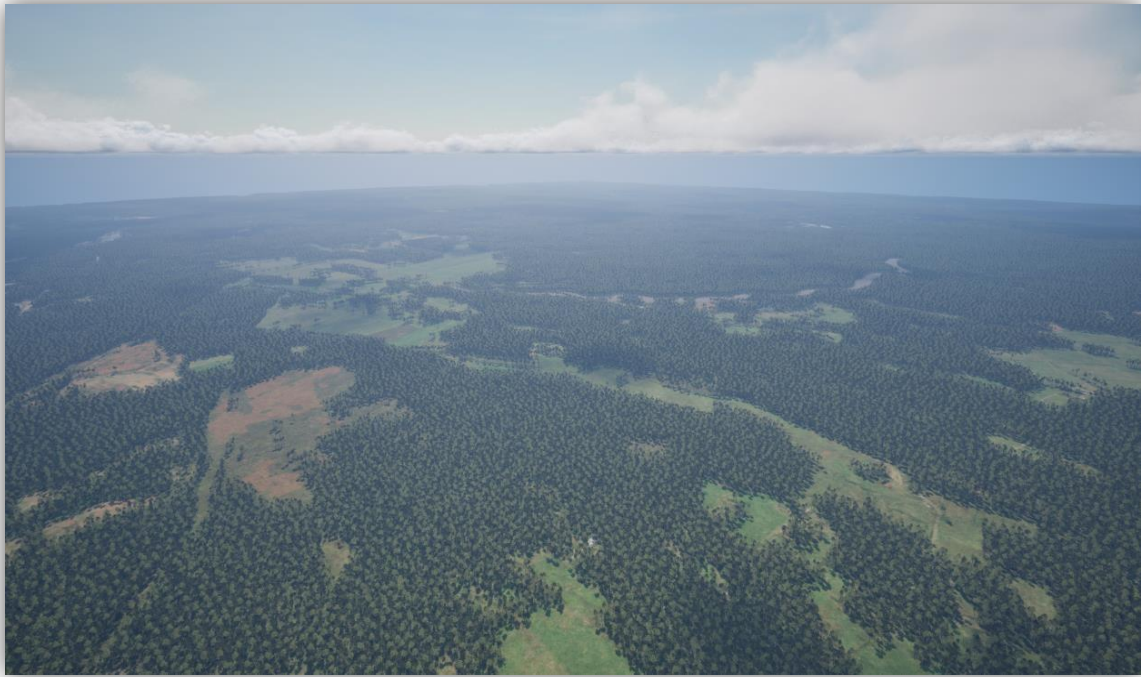


Figure 4 10.5x10.5km Landscape with environment, all of it generated using our tool and a 4033x4033 heightmap.

5.3 User Interface and Simulation Realism

A simpler custom user interface for the weather and time system was initially considered, but the decision was made to use the premade one from Ultra Dynamic Sky to save time and reduce debugging complexity. Ultra Dynamic Sky also supports the use of real coordinates and dates for accurate sun, moon, and star simulation. Enabling this feature automatically would have been an additional step to increase realism.

5.4 PCG Biomes and Landscape Coverage

A major issue encountered was the placement of a large quantity of points. Regular PCG handles this well, but PCG Biomes must calculate significantly more logic per point, making dense grass placement across a multi-kilometer landscape impractical. Many different solutions were tested, but no optimal method was found. It is possible to use PCG Biomes' runtime generation to place assets like grass, but this results in grass appearing only around the camera. For large wheat farmlands, this reduces visual realism. Combining multiple assets into larger meshes can help reduce the number of

points needed, but this may introduce other issues such as overlap. With additional time, simplification of the PCG Biomes logic could be explored, as the current system includes many features not required for this specific application.

5.5 PCG Generation Graphs

Time was spent developing custom PCG graphs to generate specific biomes. The most problematic was the water biome generator. In many games, water areas are created using a large water plane or multiple smaller planes, with terrain lowered in certain areas until the plane becomes visible. Unfortunately, these solutions do not work well in this case. Several approaches were tested without success. The main issue is that PCG Biomes does not divide the map into distinct zones, meaning that a body of water located on a hill is processed in the same graph as one at the base of the hill. As a result, using one large water plane or ensuring that smaller bodies are at the same height requires a method for grouping points that belong to the same body of water. Since this proved difficult to achieve, a less-than-perfect solution was implemented that performs adequately. The results can be seen in Figure 5



Figure 5 Image, left side: Camera within a few meters of water surface, right side: camera further than a few meters

Using this generator, the water biome maintains an acceptable visual appearance when the camera is positioned at a sufficient distance.

5.6 Development Constraints and Editor Limitations

Several unforeseen problems arose during the project, which slowed down development but also opened opportunities for future improvement. All code had to be functional within the editor rather than at runtime—an approach not previously applied in earlier projects. This introduced complications, as the editor does not clearly indicate which blueprint nodes are incompatible with editor execution.

5.7 AI-Based Segmentation of Satellite Imagery

5.7.1 Model Design and Training

A key component of the pipeline is a compact neural network trained to classify satellite pixels into biome categories. The model was implemented in PyTorch and trained on a small but diverse set of manually labeled image snippets. Each snippet was normalized and rescaled to enable fast iterations and retraining, even on modest hardware. The model's lightweight architecture (around 35,000 parameters) made it practical for real-time inference and experimentation, and it could handle high-resolution inputs up to 8129x8129 pixels without performance issues.

5.7.2 Effectiveness of the Post Processing Pipeline

The initial raw output from the AI model contained minor misclassifications, often visible as scattered patches in otherwise homogeneous regions. These inconsistencies were most prominent in edge cases or visually ambiguous terrain types. To address this, a custom post processing pipeline was developed and integrated into the toolchain.

The first stage applies morphological operations and majority filtering to remove small, isolated artifacts and smooth biome boundaries. A separate road mask is extracted from a high-contrast grayscale input, ensuring clean integration of linear features. The cleaned masks are merged into a final color-coded texture, and all outputs are resized to match the resolution of the terrain heightmap, preventing alignment issues in Unreal Engine.

This multi-stage refinement process proved effective in increasing the visual quality and usability of the segmentation output. It corrected most of the errors present in the raw AI mask and resulted in a more coherent landscape representation. The entire pipeline runs efficiently on consumer-grade hardware and requires no manual adjustment.

5.7.3 Opportunities for Refinement

Despite its flaws, the AI segmenter was sufficiently effective to drive biome placement in most test cases. The implementation offered rapid feedback and supported iterative training via an interactive developer mode. Future improvements may include expanding the labeled dataset, applying more advanced augmentation or normalization techniques, or adding a post-processing step such as morphological filtering or median smoothing to clean up noisy predictions. These changes would improve the reliability of biome classification and reduce visual inconsistencies.

5.8 Data Source Evaluation and Satellite Imagery Quality

One of the more challenging aspects during the early development phase was identifying a satellite data source that offered sufficient precision, resolution, and compatibility with the tool. Several providers—including Cesium, ArcGIS, and Unreal Map Bridge—were evaluated, but many exhibited issues such as low-resolution elevation data, restrictive licensing, or poor alignment between imagery and terrain.

The final setup involved combining a web-based PNG heightmap service (Manticorp, 2023) with a custom AI driven biome map generator. This decision was based on empirical testing, where each data source was scored based on accuracy, usability, and the amount of manual



Figure 6 Satellite imagery and heightmap used in final setup.

processing required. The evaluation process was essential for ensuring that the tool delivered realistic results with minimal user intervention. Images brought by the web-based PNG heightmap service can be seen in Figure 6.

6. Conclusion

This project demonstrates that the process of creating reality-based 3D environments from satellite data can be significantly simplified through automation and custom tooling in Unreal Engine. By combining AI-based segmentation, Python-driven data conversion, and PCG Biomes logic, we successfully built a prototype that enables users with limited 3D experience to generate detailed environments with minimal manual input. This directly answers our research question: satellite data, when integrated into a streamlined pipeline, can reduce both complexity and time in environment generation.

The tool supports scalable terrain creation, biome assignment, and weather integration, and offers advanced controls for experienced users. While current limitations exist-particularly in landscape size handling and data precision-the system lays a solid foundation for further development. Future work may include enhanced support for additional data formats, a fully integrated real-time pipeline, and performance optimization for larger and more complex environments. The findings suggest a clear path toward making photorealistic 3D reconstruction more accessible across domains that rely on geospatial data.

7. Future Work

While the current version of the tool successfully simplifies many parts of the environment creation process, it still relies heavily on manual steps. Now, users need to download satellite images and heightmaps from external websites, place them in specific folders, and run the AI segmentation tool separately. After that, they must manually import the output files into Unreal Engine and connect them to the interface. For each material used in the landscape, the user also needs to assign layer information manually and finally apply the correct masks to ensure the terrain is visually accurate.

A fully automated version of this workflow would be a major improvement. This could include built-in data download, automatic file placement, and direct integration of AI outputs into the landscape system-without requiring the user to perform each step manually. Achieving this level of automation would likely require deeper development in C++, both to optimize how Unreal handles file imports and to create better integration with the engine's landscape and material systems.

Another area for future work is performance optimization. Currently, placing a large number of assets-especially across wide landscapes-can cause slowdowns or even crashes. Improvements in how assets are placed and managed, including more efficient logic in the PCG system and the use of HLODs (Hierarchical Level of Detail), could significantly improve scalability. These types of optimizations were outside the scope of this project but offer clear potential for further exploration.

Additional user studies should be conducted to improve the user experience and simplify the usability of the tool further and to get closer to the original goal of making it usable by nearly anyone.

Finally, there is room to improve the way satellite images are processed. This could be achieved either by developing a more advanced segmentation pipeline or by continuing to refine the current AI model. Training the model on a broader and more diverse dataset, or adding additional postprocessing techniques, could lead to more precise segmentation results. Better accuracy in this step would improve the visual quality and consistency of biome placement across the generated environments.

8. References

- ChenHP (2021). *TAPython* (v1.2.4) [Unreal Engine plugin] Github
https://github.com/cgerchenhp/UE_TAPython_Plugin_Release
- delebash. (2023, August 22). *UnrealMapboxBridgePlugin* [GitHub repository]. GitHub.
<https://github.com/delebash/UnrealMapboxBridgePlugin>
- Easterday, M. W., Lewis, D. R., & Gerber, E. M. (2014). *Design-based research process: Problems, phases, and applications*. International Conference of the Learning Sciences (ICLS).
<https://repository.isls.org/handle/1/1130>
- Epic Education, Learning, and Training Staff. (2024a, October 3). *Project Atlante: An experiment in building Earth-scale worlds with UE tools only* [Video]. Epic Games.
<https://dev.epicgames.com/community/learning/talks-and-demos/DPqd/unreal-engine-project-atlantean-experiment-in-building-earth-scale-worlds-with-ue-tools-only-unreal-fest-2024>
- Epic Education, Learning, and Training Staff. (2024b, November 21). *Creating a Photoreal Environment from Scratch for Transmedia & Film/TV | Unreal Fest 2024* [Video]. Epic Games.
<https://dev.epicgames.com/community/learning/talks-and-demos/LPOY/unreal-engine-creating-aphotoreal-environment-from-scratch-for-transmedia-film-tv-unreal-fest-2024>
- Epic Games (n.d.). *Lumen Global Illumination and Reflections*. Epic Developer Community
<https://dev.epicgames.com/documentation/en-us/unreal-engine/lumen-global-illumination-and-reflections-in-unreal-engine>
- Epic Games (n.d.). *Nanite Virtualized Geometry*. Epic Developer Community
<https://dev.epicgames.com/documentation/en-us/unreal-engine/nanite-virtualized-geometry-in-unreal-engine>
- Esri. (2024, August). *ArcGIS Maps SDK for Unreal Engine*. Esri.
<https://www.fab.com/listings/6bbabfe7-4c9d-484a-a97b-45a9215139cc>
- Frenzel, E. (2025, April 11). *Landscape AutoMaterial w/ Satellite Image Overlay*. Frenzel.
<https://www.fab.com/listings/8d03a919-9d25-471f-b040-818947fb0c57>
- Gross, F., Petersen, L., Wallmeier, C., & Karstens, S. (2022). *Coastscapes for a Metaverse: From height maps to virtual vegetated environments*. ResearchGate.
<https://doi.org/10.21203/rs.3.rs-2225992/v1>
- Gunther, E. (2025, April 16). *Ultra Dynamic Sky*. Gunther. <https://www.fab.com/listings/84fda27a-c79f-49c9-8458-82401fb37cfb>
- Manticorp. (2023, March 31). *Unreal Heightmap Generator* [GitHub repository]. GitHub.
<https://github.com/manticorp/unrealheightmap>
- Meta AI. (2024, December 11). *Segment Anything Model 2 (SAM2)* [GitHub repository]. GitHub.
<https://github.com/facebookresearch/sam2>

Misztal, M. (2024, January 23). *Creating Open World Environments in Unreal Engine 5: Part 1*. The Rookies. <https://discover.therookies.co/2024/01/23/creating-open-world-environments-in-unreal-engine-5part-1/>

NumPy Developers. (2023). *NumPy*. <https://numpy.org/>

OpenAI. (n.d.). *OpenAI o1* [Web page]. <https://openai.com/o1/>

OpenCV Team. (2023). *OpenCV Library*. <https://opencv.org/>

Pillow Developers. (2023). *Pillow* <https://pillow.readthedocs.io/> PyTorch Developers. (2023). *PyTorch*. <https://pytorch.org/>

Python Software Foundation. (2023). *hashlib* <https://docs.python.org/3/library/hashlib.html>

Python Software Foundation. (2023). *pickle* <https://docs.python.org/3/library/pickle.html>

sanford87. (2020, October 10). *Terrain recreation from satellite imagery similar to Microsoft Flight Simulator 2020* [Forum post]. Unreal Engine Forums. <https://forums.unrealengine.com/t/terrain-recreation-from-satellite-imagery-similar-to-microsoftflight-simulator-2020/151962>

scikit-learn Developers. (2023). *scikit-learn: Machine Learning in Python*. <https://scikit-learn.org/>

Torchvision Developers. (2023). *Torchvision*. <https://pytorch.org/vision/stable/index.html>

9. Appendix

Method	Tested / Not Tested	Heightmap	Biomemap	Satellite Image	Masks	Total Score	Comment	Link
Cesium	Tested	Difficult to get accurate heightdata	Not supported	Decent quality, some tiling.	Not Supported	2.25/5	Could work but requires separate calculations	L
ArcGis	Tested	Has Heightdata	Missing data in some areas	Alot of different formats but hard to access	Not Supported	2.5/5	Difficult to use and seems to have some major bugs	L
BlackSharkAi	Not Tested	Has Heightdata	Possible to train the AI to detect different ground types	Seems to have high quality images	Likely possible		Not publicly available but potentially very useful	L
Unreal Map Bridge	Tested	Good Quality Heightmaps that work with Unreal Engine	Possible to get low precision biomemap	Good quality, doesn't match other data	Possible to download masks based on biomemap	3.5/5	Good potential but the mismatching data makes it difficult to use	L
Unreal Png Heightmap	Tested	Good Quality Heightmaps that work with Unreal Engine	Not supported	Good quality images	Not Supported	3.0/5	Simple and works but is slightly limited in the maps zoom level	L

Clarification:

We graded every category separately based on what we needed for this project versus what we got but decided to only include our Total Score to be able to fit the table on this page easily.

AI-Model	Tested / Not Tested	Precision	Pros	Cons	Comment
SAM-2	Tested with assistance of GPT o1	★★★★☆	Has high precision	Difficult to use without sufficient knowledge	Works well according to others, we were not able to get it to work for our needs
Custom model in PyTorch (SimpleMLP)	Tested with assistance of GPT o1	★★★★☆	Easy to use with decent precision	Less accurate	You train the AI yourself, suits what we need

Clarification:

The entry "PyTorch/Torchvision" refers not to a pre-trained model, but to the underlying frameworks used to build and train our own neural network. The model architecture (SimpleMLP) and the training procedure were developed entirely in-house. As such, this row in the table represents a custom AI solution implemented using PyTorch-not a pre-existing AI model like SAM-2.